

Data Structure Lab7 : Stack 2022-2023

Topics

1. Create Stack Interface
2. Create Stack Using Array
3. Create Stack Using Linked Lists
4. Implement Basic Methods of Stack
 - isEmpty()
 - size()
 - top()
 - push(E e)
 - pop()

Homework

1. Implement a method with signature transfer(S, T) that transfers all elements from stack S onto stack T, so that the element that starts at the top of S is the first to be inserted onto T, and the element at the bottom of S ends up at the top of T.

```
public void transfer(Stack<E> S, Stack<E> T) {  
    Stack<E> temp = new ArrayStack<>();  
    while (!S.isEmpty()) {  
        temp.push(S.pop());  
    }  
    while (!temp.isEmpty()) {  
        T.push(temp.pop());  
    }  
}
```

2. Give a recursive method for removing all the elements from a stack.

Data Structure Lab7 : Stack 2022-2023

3. Postfix notation is an unambiguous way of writing an arithmetic expression without parentheses. It is defined so that if “(exp1)op(exp2)” is a normal fully parenthesized expression whose operation is op, the postfix version of this is “pexp1 pexp2 op”, where pexp1 is the postfix version of exp1 and pexp2 is the postfix version of exp2. The postfix version of a single number or variable is just that number or variable. So, for example, the postfix version of “((5 + 2) * (8 – 3))/4” is “5 2 + 8 3 – * 4 /”. Describe a nonrecursive way of evaluating an expression in postfix notation.

-Create a stack to hold operands.

-For each token in the expression:

- If the token is an operand (number or variable), push it onto the stack.
- If the token is an operator (e.g., +, -, *, /):
 - Pop the top two operands from the stack.
 - Perform the operation.
 - Push the result back onto the stack.

4. Implement the clone() method for the ArrayStack class.

```
public ArrayStack<E> clone() {  
    ArrayStack<E> newStack = new ArrayStack<>(this.size());  
    for (int i = 0; i < this.size(); i++) {  
        newStack.push(this.array[i]);  
    }  
    return newStack;  
}
```

5. Implement a program that can input an expression in postfix notation (see Exercise C-6.19) and output its value

Data Structure Lab7 : Stack 2022-2023

```
public class PostfixEvaluator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a postfix expression: ");
        String expression = scanner.nextLine();
        System.out.println("Value: " + evaluatePostfix(expression));
    }

    private static double evaluatePostfix(String expression) {
        Stack<Double> stack = new ArrayStack<>();
        String[] tokens = expression.split(" ");

        for (String token : tokens) {
            if (isOperator(token)) {
                double b = stack.pop();
                double a = stack.pop();
                stack.push(applyOperator(a, b, token));
            } else {
                stack.push(Double.parseDouble(token));
            }
        }
        return stack.pop(); // Final result
    }

    private static boolean isOperator(String token) {
        return "+-*/".contains(token);
    }

    private static double applyOperator(double a, double b, String
operator) {
        switch (operator) {
            case "+":
                return a + b;
        }
    }
}
```

Data Structure Lab7 : Stack 2022-2023

```
        case "-":
            return a - b;
        case "*":
            return a * b;
        case "/":
            return a / b;
        default:
            throw new IllegalArgumentException("Invalid operator: " +
operator);
    }
}
```